Luma

Bright Living, Smart Control

Detailed Presentation Script

With Project File Descriptions and Corresponding UI Sections

Module Code:	SCS 3311
Module Name:	Mobile Application Design and Development
Project Type:	Mini Project
Platform:	Android (Kotlin) · Firebase · React Simulator

Presentation Members

Member 1	Android UI, Navigation, Dashboard, Authentication
Member 2	Firebase, Realtime Database, Synchronization, Device Data
Member 3	React Simulator, Cloud Functions, Testing, Documentation

August 14, 2026

Contents

1 Presentation Overview

1.1 Opening

*Good morning/afternoon. Today we are presenting **Luma**, a Smart Home Monitoring and Control System developed for the module SCS 3311 – Mobile Application Design and Development.*

Luma allows a user to monitor and control smart home devices such as lights, outlets, irons, switch panels, and cameras. Since we did not use physical IoT hardware for this mini project, we built a separate web-based hardware simulator. The simulator behaves like virtual smart devices and synchronizes with the Android application through Firebase Realtime Database.

1.2 Project Structure

The project is organized as a monorepo. The `android/` folder contains the Android mobile application. The `simulator/` folder contains the React hardware simulator. The `firebase/` folder contains Cloud Functions and database rules. The `docs/` folder contains the report, schema documentation, and presentation material.

Folder	Description
`android/`	Native Android application built using Kotlin, Jetpack Compose, Firebase Authentication, and Firebase Realtime Database.
`simulator/`	React and Vite web application that simulates smart home hardware and updates Firebase in real time.
`firebase/`	Firebase project configuration, Realtime Database rules, and Node.js Cloud Functions for automation.
`docs/`	Academic documentation, project report, database schema, and presentation script.

Table 1: Main project folder structure.

1.3 Architecture Summary

The Android app and the React simulator do not communicate directly with each other. Both are independent clients of the same Firebase Realtime Database. Firebase acts as the single source of truth. When the Android app changes a device state, the simulator receives the update through Firebase listeners. Similarly, when the simulator changes a device state, the Android app receives the update through its Firebase listener.

Firebase Cloud Functions provide backend automation. For example, the iron auto-shutdown function checks whether an iron has been on for too long and automatically turns it off for safety.

2 File and UI Correspondence Summary

Project File	Corresponding UI / Feature	Purpose
`android/.../MainActivity.kt`	Android navigation root	Launches the app, applies the theme, and defines navigation routes.
`android/.../WelcomeScreen.kt`	Welcome UI	Shows welcome image, title, tagline, Sign In button, and Sign Up button.
`android/.../SignInScreen.kt`	Login UI	Provides email/password login and calls Firebase authentication logic.
`android/.../SignUpScreen.kt`	Registration UI	Collects user details and creates a new account.
`android/.../welcomePage.kt`	Auth controller functions	Contains `registration()` and `login()` functions using Firebase Auth and Realtime Database.
`android/.../ValidationUtils.kt`	Form validation	Validates registration inputs before account creation.
`android/.../DashboardScreen.kt`	Mobile dashboard	Shows user greeting, energy card, floor cards, bottom navigation, and add-floor dialog.
`android/.../FloorDetailsScreen.kt`	Floor device list	Shows devices for a selected floor and opens camera feed for camera devices.
`android/.../CameraFeedScreen.kt`	Mobile camera feed	Displays camera snapshot, live/offline state, and last snapshot time.
`android/.../ui/theme/*.kt`	Android styling	Defines colors, typography, and app theme.
`simulator/src/App.tsx`	Simulator app shell	Sets up routing, theme, Firebase auth state, navbar, sidebar, and page layout.
`simulator/src/pages/LoginPage.tsx`	Simulator login UI	Authenticates simulator users through Firebase Authentication.
`simulator/src/pages/FloorOverviewPage.tsx`	Simulator dashboard	Shows selected floor, rooms, device cards, summary stats, and add/delete controls.
`simulator/src/components/DeviceCard.tsx`	Device card UI	Displays each device with icon, status chip, and control actions.

Project File	Corresponding UI / Feature	Purpose
`simulator/src/pages/DeviceDetailPage.tsx`	Simulator device details	Shows device-specific controls such as schedules, switch panel controls, iron timer, and error simulation.
`simulator/src/pages/CameraFeedPage.tsx`	Simulator camera feed	Shows mock camera snapshot and Capture Snapshot button.
`simulator/src/pages/LogsPage.tsx`	Logs, reports, alerts	Shows activity logs, device history, reports, and alert events.
`simulator/src/pages/ProfilePage.tsx`	Profile UI	Displays account information read from Firebase.
`simulator/src/firebase/firebaseConfig.ts`	Firebase config	Initializes Firebase app, auth, and database connection using environment variables.
`simulator/src/firebase/deviceService.ts`	Shared data service	Contains functions for floors, rooms, devices, toggles, schedules, snapshots, and errors.
`firebase/functions/src/index.js`	Cloud Function exports	Exports all backend automation functions.
`firebase/functions/src/ironAutoShutdown.js`	Iron safety automation	Automatically turns off irons after the configured duration.
`firebase/functions/src/lightScheduler.js`	Light scheduling	Turns lights on/off based on saved time schedules.
`firebase/functions/src/disconnectionDetection.js`	Connection monitoring	Marks devices as disconnected when heartbeat timestamps are too old.
`firebase/functions/src/reportAggregator.js`	Usage report generation	Calculates running time and estimated energy usage when devices turn off.
`firebase/functions/src/alertNotifier.js`	Notification handling	Processes new alert entries and logs alert information.
`firebase/database.rules.json`	Database security	Defines Realtime Database read/write security rules.

Table 2: Project files mapped to their corresponding UI sections and features.

3 Android Application Script

3.1 Android Entry Point and Navigation

Now I will explain the Android application. The Android application starts from `MainActivity.kt`*. This file is the entry point of the mobile app. It enables edge-to-edge display, applies the* `LumaaTheme`*, and calls the* `AppNavigation()` *composable.*

Inside `AppNavigation()`, the app uses Jetpack Navigation Compose. The main routes are `welcome`, `signin`, `signup`, `dashboard`, `floor_details/{floorId}`, and `camera_feed/{deviceId}`. This navigation flow allows the user to move from the welcome screen to authentication, then to the dashboard, then to floor details and camera monitoring.

3.2 Welcome Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/WelcomeScreen.kt`

The first UI section is the Welcome Screen. This screen displays the welcome image, the title "Welcome Lumaa", the subtitle "Make your home smarter", and two main buttons: Sign In and Sign Up. It introduces the purpose of the app before the user logs in or creates an account.

Corresponding UI elements:

- Welcome image from Android resources.
- Main title: `Welcome Lumaa`.
- Subtitle: `Make your home smarter`.
- **Sign In** button.
- **Sign Up** button.
- Description text explaining that Luma helps users control, monitor, and automate their smart home.

3.3 Sign In Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/SignInScreen.kt`

The Sign In Screen is used for user login. It contains an email field, a password field, and a Login button. If either field is empty, a snackbar message asks the user to fill all fields. When the user enters valid credentials, the screen calls the `login()` function and navigates to the dashboard after successful authentication.

Corresponding UI elements:

- Top app bar with back button.
- Email input field.
- Password input field with password masking.
- Login button.
- Loading indicator while authentication is running.
- Snackbar for validation and login errors.

3.4 Sign Up Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/SignUpScreen.kt`

The Sign Up Screen is used to create a new Luma account. This screen collects full name, email address, phone number, home address, and password. Before creating the account, the app validates the input using `ValidationUtils.kt`. If validation passes, it creates a Firebase Authentication account and stores the profile details in Firebase Realtime Database.

Corresponding UI elements:

- Full Name input field.
- Email Address input field.
- Phone Number input field.
- Home Address input field.
- Password input field.
- Sign Up button.
- Loading indicator.
- Snackbar messages for errors and success.

3.5 Authentication Controller

File: `android/app/src/main/java/com/example/myapplication/ui/pages_controllers/welcomePage.kt`

The authentication logic is handled in `welcomePage.kt`. The `registration()` function creates a new user with Firebase Authentication and then writes user data such as name, email, phone, address, and role into the `users` node of Firebase Realtime Database. The `login()` function signs in the user using Firebase Authentication.

3.6 Dashboard Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/DashboardScreen.kt`

After login, the user reaches the Dashboard Screen. This is the main smart home control screen in the Android app. The dashboard reads the user name and floor data from Firebase. It shows a welcome message, an estimated energy card, a Smart Home Controller section, floor cards, a bottom navigation bar, and a floating action button.

Each floor card represents a floor of the home. The card shows the floor icon, floor name, a short description, and a switch. The user can tap a floor card to open the Floor Details Screen. The floating action button opens a dialog to add a new floor. If the user edits an existing floor, the same dialog can update the floor name and icon.

Corresponding UI elements:

- Navigation drawer with Profile, Settings, and Logout options.
- Top app bar with menu and profile icon.
- Greeting section: `Welcome Home` and user name.
- Estimated energy card.
- `Smart Home Controller` heading.
- Floor cards displayed in a two-column grid.
- Floor switch to update `isOn`.
- Floating action button for adding floors.
- Add/Edit Floor dialog with floor name and icon selector.

3.7 Floor Details Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/FloorDetailsScreen.kt`

When the user taps a floor card, the app navigates to the Floor Details Screen. This screen receives the selected `floorId` from the navigation route. It reads the selected floor name and its related device IDs from Firebase. Then it loads the device details from the global `devices` node.

The UI shows a top app bar with the floor name and the subtitle "Smart Devices". If the data is loading, a progress indicator is displayed. If no devices exist, the app shows a "No devices found" message. Otherwise, the devices are shown in a list.

Corresponding UI elements:

- Back button.
- Floor name title.
- `Smart Devices` subtitle.
- Loading progress indicator.
- Empty state for no devices.
- Device rows with icon, name, and status.
- `View Feed` action for camera devices.

3.8 Camera Feed Screen

File: `android/app/src/main/java/com/example/myapplication/ui/screens/CameraFeedScreen.kt`

The Camera Feed Screen is opened when the user selects a camera device. It receives the `deviceId` *from navigation and listens to* `devices/{deviceId}` *in Firebase. It reads the camera name, status, last snapshot timestamp, and Base64 snapshot image.*

If there is no snapshot, the screen shows a "No snapshot yet" message. If a snapshot exists, the app decodes the Base64 image and displays it as the camera preview. At the bottom of the preview, the app shows whether the camera is LIVE or OFFLINE and displays the last snapshot time.

Corresponding UI elements:

- Back button.
- Camera name.
- Camera Feed subtitle.
- Snapshot image preview.
- LIVE/OFFLINE indicator.
- Last snapshot timestamp.
- Empty state when no snapshot is available.

4 React Hardware Simulator Script

4.1 Simulator Purpose

Now I will explain the React hardware simulator. The simulator is a separate web application located in the `simulator/` *folder. It is not part of the Android app. Its purpose is to replace physical IoT hardware during development and demonstration.*

When a device is toggled in the simulator, the simulator writes the updated state to Firebase. The Android app receives the change in real time. From Firebase's point of view, simulator data behaves like data from real IoT hardware.

4.2 Simulator App Shell

File: `simulator/src/App.tsx`

The main simulator file is `App.tsx`*. This file creates the Material UI theme, checks Firebase authentication state, sets up logging, and defines the React routes. It also renders the Navbar, Sidebar, footer, and the main page area.*

Corresponding UI sections:

- Login state handling.

- Top navigation bar.
- Sidebar floor selector.
- Main content area.
- Routes for floor overview, device details, camera feed, logs, and profile.
- About dialog and footer.

4.3 Simulator Login Page

File: `simulator/src/pages/LoginPage.tsx`

The simulator also has a login page. This page shows the Luma Hardware Simulator login form. It contains email and password fields, a password visibility toggle, error messages, and a login button. It authenticates the user through Firebase Authentication.

4.4 Floor Overview Page

File: `simulator/src/pages/FloorOverviewPage.tsx`

The Floor Overview Page is the main simulator dashboard. It shows the selected floor, rooms, device cards, and summary statistics. It also allows users to add rooms, add devices, delete rooms, delete devices, collapse room sections, and toggle devices.

This page listens to Firebase using `onValue`. When data changes in Firebase, the simulator updates automatically. The page also calculates total devices, active devices, and error devices for the selected floor.

Corresponding UI elements:

- Selected floor header.
- Summary stat cards.
- Room sections.
- Add Room button.
- Add Device button.
- Delete room/device confirmation dialogs.
- Device cards.
- Empty and loading states.

4.5 Device Card Component

File: `simulator/src/components/DeviceCard.tsx`

The Device Card component is a reusable UI component used inside the simulator dashboard. Each device card displays the device icon, device name, type, current status, and control actions. This is the simulator's main device-level UI section.

4.6 Device Detail Page

File: `simulator/src/pages/DeviceDetailPage.tsx`

The Device Detail Page shows detailed controls for a selected device. It changes its UI depending on the device type. For normal devices, it shows power control. For smart lights, it shows schedule fields. For switch panels, it shows individual switch controls. For irons, it shows an auto-shutdown timer slider. For cameras, it provides a link to the camera feed.

Corresponding UI sections:

- Device header with icon, name, and type.
- Status and Power card.
- Auto Schedule card for smart lights.
- Switches card for switch panel devices.
- Auto Shutdown Timer card for iron devices.
- Snapshot card for camera devices.
- Simulator Controls card with Simulate Error button.

4.7 Camera Feed Page

File: `simulator/src/pages/CameraFeedPage.tsx`

The simulator Camera Feed Page displays a mock camera snapshot. It shows the camera name, live/offline indicator, last snapshot time, and a Capture Snapshot button. When the Capture Snapshot button is clicked, the simulator requests a new mock image and writes the snapshot information to Firebase. The Android camera screen can then display the same updated image.

4.8 Logs Page

File: `simulator/src/pages/LogsPage.tsx`

The Logs Page displays activity logs, device history, reports, and alerts. For example, page navigation, device toggles, switch toggles, device running history, estimated energy usage, and alert events can be shown here. This page is important for demonstrating monitoring and reporting features.

4.9 Profile Page

File: `simulator/src/pages/ProfilePage.tsx`

The Profile Page displays user account information such as name, email, phone number, address, user ID, and role. It reads this information from Firebase and presents it in a clean account details layout.

4.10 Simulator Firebase Service

File: `simulator/src/firebase/deviceService.ts`

The file ***`deviceService.ts`*** *contains the main Firebase operations used by the simulator. It includes functions to get floors, get devices, add floors, add rooms, add devices, delete rooms, delete devices, toggle device state, update switch state, update light schedules, update iron max duration, request camera snapshots, trigger device errors, and seed sample data.*

Important service functions:

- `setDeviceState()` updates device `state`, `status`, and `lastSeen`.
- `setSwitchState()` updates an individual switch inside a switch panel.
- `updateDeviceSchedule()` saves light start and end times.
- `updateIronMaxDuration()` saves the iron auto-shutdown duration.
- `requestCameraSnapshot()` generates and stores a mock camera snapshot.
- `triggerDeviceError()` sets a device status to `ERROR`.
- `seedSampleData()` creates sample floors, rooms, and devices for a new user.

5 Firebase Backend Script

5.1 Firebase Configuration

Firebase is the central backend of Luma. It provides Authentication, Realtime Database, and Cloud Functions. The Android app and simulator both connect to Firebase. Firebase Realtime Database stores users, floors, rooms, devices, alerts, reports, and device history.

5.2 Database Rules

File: `firebase/database.rules.json`

The database rules file controls read and write permissions for Firebase Realtime Database. In a Firebase-based system, security rules are important because clients connect directly to the database instead of going through a custom backend API.

5.3 Cloud Function Entry File

File: `firebase/functions/src/index.js`

The file ***`index.js`*** *exports all Firebase Cloud Functions used in the project. These functions are* ***`ironAutoShutdown`****,* ***`disconnectionDetector`****,* ***`lightScheduler`****,* ***`reportAggregator`****, and* ***`alertNotifier`****.*

5.4 Iron Auto-Shutdown Function

File: `firebase/functions/src/ironAutoShutdown.js`

The iron auto-shutdown function is a safety automation feature. It runs every minute and checks all devices. If a device is an iron, if its state is ON, and if it has been on longer than its configured maximum duration, the function automatically sets the iron state to OFF. It also clears the `turnedOnAt` *value, updates* `lastSeen`*, and creates an alert.*

Corresponding UI section: Auto Shutdown Timer card in `DeviceDetailPage.tsx`.

5.5 Light Scheduler Function

File: `firebase/functions/src/lightScheduler.js`

The light scheduler function checks smart light devices every minute. It compares the current time with the saved `startTime` *and* `endTime`*. If the current time is inside the schedule, the light is turned ON. Otherwise, it is turned OFF.*

Corresponding UI section: Auto Schedule card for light devices in `DeviceDetailPage.tsx`.

5.6 Disconnection Detector Function

File: `firebase/functions/src/disconnectionDetector.js`

The disconnection detector checks whether devices have stopped updating their `lastSeen` *timestamp. If a device has not been seen for more than the threshold, the function marks it as* `DISCONNECTED` *and creates an alert.*

Corresponding UI sections: device status chips, dashboard error count, and logs/alerts page.

5.7 Report Aggregator Function

File: `firebase/functions/src/reportAggregator.js`

The report aggregator runs when a device changes from ON to OFF. It calculates how long the device was on, estimates energy consumption using nominal wattage values, writes a history entry, and updates the report data.

Corresponding UI section: device history and reports tabs in `LogsPage.tsx`.

5.8 Alert Notifier Function

File: `firebase/functions/src/alertNotifier.js`

The alert notifier runs when a new alert is created under the `/alerts` *node. It reads the related device name and logs the alert type, message, device, and timestamp. This supports safety and monitoring events in the system.*

6 Demonstration Flow

6.1 Step 1: Show Welcome and Authentication

1. Open the Android app.
2. Show the Welcome Screen.
3. Tap Sign In and show the login form.
4. Optionally show Sign Up and explain user registration fields.
5. Log in and navigate to the dashboard.

6.2 Step 2: Show Android Dashboard

1. Show the greeting and user name.
2. Point to the estimated energy card.
3. Show the Smart Home Controller section.
4. Explain the floor cards.
5. Tap the floating action button and show the Add Floor dialog.
6. Tap a floor card to open Floor Details.

6.3 Step 3: Show Floor Details and Camera Feed

1. Show device rows for the selected floor.
2. Explain the device icons and status text.
3. Tap a camera device.
4. Show the Camera Feed Screen.
5. Explain live/offline status and snapshot timestamp.

6.4 Step 4: Show Simulator Dashboard

1. Open the React simulator.
2. Log in using Firebase credentials.
3. Show the sidebar floor list.
4. Select a floor and show room/device sections.
5. Toggle a device and show realtime update in Firebase/Android.
6. Open a device details page and show type-specific controls.

6.5 Step 5: Demonstrate Device-Specific Features

1. For a light, show schedule start and end time.
2. For a switch panel, toggle individual switches.
3. For an iron, adjust the max duration slider and explain auto-shutdown.
4. For a camera, capture a new snapshot.
5. Use Simulate Error to show an error status.

6.6 Step 6: Show Logs, Reports, and Alerts

1. Open the simulator Logs page.
2. Show user activity logs.
3. Show device history.
4. Show report entries.
5. Show alert events generated by automation or simulated errors.

6.7 Step 7: Explain Firebase Backend

1. Open Firebase Realtime Database.
2. Show `users`, `floors`, `devices`, `alerts`, and `reports`.
3. Explain that devices are stored globally and referenced from rooms.
4. Explain Cloud Functions for safety automation and reports.

7 Closing Script

To conclude, Luma demonstrates a complete smart home monitoring and control system using Android, React, Firebase Realtime Database, and Firebase Cloud Functions. The Android app provides the mobile user interface. The React simulator replaces physical IoT hardware. Firebase connects both clients in real time and provides backend automation.

Through this project, we demonstrated mobile application development using Kotlin and Jetpack Compose, web-based hardware simulation using React, realtime synchronization using Firebase, and automation using Firebase Cloud Functions. Luma is therefore a practical and testable smart home system suitable for our mobile application development assignment. Thank you.

8 Quick Presentation Checklist

#	Item	Done
1	Introduce project name, module, and purpose.	☐
2	Explain Android, simulator, and Firebase architecture.	☐
3	Show Android Welcome, Sign In, and Sign Up screens.	☐
4	Show Android Dashboard and floor cards.	☐
5	Show Android Floor Details and Camera Feed screens.	☐
6	Show simulator login and floor overview dashboard.	☐
7	Show simulator device detail controls.	☐
8	Demonstrate realtime synchronization through Firebase.	☐
9	Explain `deviceService.ts` Firebase operations.	☐
10	Explain Cloud Functions and their corresponding UI sections.	☐
11	Show logs, reports, and alerts in the simulator.	☐
12	Finish with conclusion and future potential.	☐

Table 3: Presentation checklist.